International Journal of Intelligent Systems

WILEY

RESEARCH ARTICLE OPEN ACCESS

Comparative Evaluation of ChatGPT and DeepSeek for Competitive Programming: International Collegiate Programming Contest Case

Harshita Vyas[1] | Ravindra G. Bhardwaj[2,3]

[1]Department of Computer Science, Stanley College of Engineering and Technology for Women, Hyderabad, Telangana, India | [2]Department of Mechanical Engineering, Birla Institute of Technology and Science Pilani, Dubai Campus, Dubai International Academic City, Dubai, UAE | [3]Department of Physics, Birla Institute of Technology and Science, Hyderabad Campus, Jawahar Nagar, Kapra Mandal, Medchal District, Pilani 500078, Telangana, India

Correspondence: Ravindra G. Bhardwaj (ravindra.waj@gmail.com)

Received: 28 August 2025 | **Revised:** 23 December 2025 | **Accepted:** 31 December 2025

Academic Editor: Kallol Mohanta

Keywords: ChatGPT | code generation | code quality | DeepSeek | education | ICPC

ABSTRACT

The International Collegiate Programming Contest (ICPC) is widely regarded as one of the most prestigious algorithmic programming competitions for university students. Given the challenges faced by students from developing countries in preparing for the contest, it is important to examine how generative AI tools can support their learning and preparation. The effectiveness of two leading generative AI models, ChatGPT and DeepSeek, is evaluated in addressing complex programming problems based on the Association for Computing Machinery (ACM)'s International Collegiate Programming Contest (ICPC) (ACM ICPC). The evaluation of both models in terms of readability, error handling, computation speed, code accuracy, and educational value is presented in this study. In a two-trial experimental setup, both models are evaluated on 145 different ICPC problems from data structures, algorithms, mathematics, geometry, advanced optimization, and so on. Prompts were standardized, and evaluation was conducted over two iterations to simulate iterative learning. The results indicate that both DeepSeek and ChatGPT improved their performance over time. DeepSeek consistently outperformed ChatGPT in code accuracy (88.28% vs. 84.14%), both generated more efficient algorithms for linear time complexity (41 vs. 19), and had lower logical error rates (7.58% vs. 15.86%). DeepSeek and ChatGPT performed almost the same in code quality scores (37.79 vs. 37.85). Approximately 46.90% of the solutions generated by DeepSeek were fully insightful, surpassing ChatGPT's 42.07%. However, ChatGPT demonstrated significant improvement across trials, particularly drastically reducing syntax errors from 4.83% to 0.69%. DeepSeek outperforms ChatGPT in high-stakes programming scenarios, making it the more suitable choice. These results offer actionable guidance for incorporating generative AI tools into advanced programming education.

1 | Introduction

Generative artificial intelligence (GenAI) creates new content, such as text, images, or music, based on patterns in the data it has been trained on. GenAI utilizes large language models (LLMs), which involve training on extensive text datasets and employing AI models with a large number of model parameters [1, 2]. With the rapid adoption of AI models for code generation [3, 4], GenAI has emerged as a transformative force in computer education. Tools based on GenAI, such as ChatGPT, have quickly become commonplace in education [5], particularly in tasks like

This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

Copyright © 2026 Harshita Vyas and Ravindra G. Bhardwaj. *International Journal of Intelligent Systems* published by John Wiley & Sons Ltd.

programming [6]. ChatGPT and DeepSeek represent the latest frontier in AI-powered assistance, combining natural language processing with domain-specific knowledge to help users write, debug, and comprehend code. However, there is no study on the competitive behavior of two well-known GenAI tools, ChatGPT and DeepSeek, in the context of the prestigious ACM ICPC algorithmic programming competition. Accordingly, this study conducts a systematic performance evaluation of ChatGPT and DeepSeek by compiling, categorizing, and analyzing the complete set of ACM ICPC programming problems from the past 11 years.

ChatGPT has gained popularity in various aspects due to its flexibility and human-like interaction, in particular. Educational settings have begun using GenAI for code generation, an area of great interest [7]. Researchers and industry have taken notice of the recently released DeepSeek [8]. This well-known open-source tool has grown in popularity for AI modeling, giving performance that is competitive but with inference costs [9]. DeepSeek-Coder, a specialized LLM custom-made for coding, gives you open-source and performance improvement for difficult programming jobs [10]. The rise of these tools has sparked an academic debate on their effectiveness and limitations in high-stakes educational contexts, especially in programming engineering dimensions to a varied degree [11]. Recent research studies highlight the growing significance of GenAI tools in programming education and software development. GPT-3.5 can generate mostly accurate programming exercises in several major languages but performs poorly in Tamil, emphasizing the need for further multilingual evaluation and the potential of LLMs to support non-native English speakers in computer science education [12]. Studies show that models like ChatGPT enhance student engagement, debugging capabilities, and learning outcomes in coding tasks through natural language explanations and real-time feedback [13]. Comparative evaluations with traditional learning methods suggest that GenAI improves comprehension of algorithms and data structures [14, 15]. Educators are increasingly integrating these tools into classrooms for automated grading, problem generation, and concept clarification [16]. Furthermore, GenAI models trained on domain-specific corpora exhibit higher code accuracy and reduced syntactic errors than general-purpose ones [17]. Despite limitations in logical reasoning, GenAI excels at generating readable, modular code with useful documentation. A recent case study on the transformative potential of AI in software engineering using LeetCode and ChatGPT explores how prompt engineering significantly influences model performance in competitive coding contexts [18, 19]. Recent studies have examined GenAI models for programming education and code generation; however, very few have investigated how these models perform in competitive programming contests, such as the ICPC. There is little understanding of how models handle different problem types, difficulty levels, or competition-style constraints. This gap motivates our research questions, which focus on evaluating code correctness, efficiency, reasoning, and errors in ChatGPT and DeepSeek.

This study compares ChatGPT and DeepSeek on a challenging benchmark: the Association for Computing Machinery (ACM)'s International Collegiate Programming Contest (ICPC). The ICPC is one of the world's most prestigious algorithmic programming competitions for university students, organized by the ACM. It features a multitiered structure comprising local and regional contests, culminating in the World Finals, which invite only the top ~1% of teams globally. In these finals, teams of three students are given 5 h to solve 8–13 challenging problems on a single computer, assessing a range of topics. With only the top 1% of global teams advancing to the world finals, this world-renowned exam evaluates the problem-solving skills required for competitive programming and software code engineering. Participation in the ICPC world finals is widely recognized as a mark of distinction in the field of computer science. This competition serves as a powerful motivator for students and entry-level professionals to refine their programming skills under timed conditions and realistic problem contexts [20]. Beyond skill development, success in ACM/ICPC is widely regarded as an indicator of high intellectual aptitude. Notably, many technical job interviews incorporate algorithmic challenges that closely resemble those encountered in such competitions, giving experienced candidates a competitive edge in the recruitment process and further enhancing their employability. Corporate sponsors, such as IBM, not only fund contest prizes but also actively recruit medalists through internships and direct employment pathways, often prioritizing gold-winning team members for technical roles. Moreover, the career benefits of contest participation extend beyond the private sector. Universities and academic institutions frequently hire accomplished ACM/ICPC participants as programming instructors or team coaches, creating additional employment avenues in education and research. Taken together, these illustrate why students are increasingly motivated to engage in the programming competition [21].

In this study, we employ a dual-trial framework for conducting a comprehensive evaluation of ChatGPT (GPT-4o version) and DeepSeek (V3.2) by testing their performance on past ACM ICPC World Finals problems from the years 2013–2024 that were officially available. The models were tested on 145 questions covering a wide spectrum of topics by initial code generation attempts, followed by model fine-tuning using indirect training, with solutions executed and tested in a standardized C++ environment. Beyond quantifying accuracy, we assess technical code quality through execution-based metrics (syntax/logical/runtime errors) and structural analysis, including overall readability scores derived from comment density, identifier quality, indentation score, and function size. This approach bridges the gap between theoretical code generation capabilities and real-world programming competition demands by providing actionable insights into how GenAI tools can complement competitive programming training. Our methodology ensures a robust and fair assessment of each model's strengths and limitations in competitive programming contexts. Overall, this study analyzes the performance of ChatGPT and DeepSeek across varied difficulty levels and topics, including data structures, algorithms, mathematics, computational geometry, optimization, bit manipulation, and string manipulation. This research highlights the strengths and limitations of each model in solving competitive coding tasks, providing insights into their practical utility in advanced programming education and assessments. A preprint has previously been published [22].

2 | Background

Given the rapid adoption of AI in education, it has become increasingly important to evaluate how effectively GenAI tools

perform in rigorous academic settings. Although AI originated within computer science, its adoption in education has accelerated rapidly, with AI now being integrated across various disciplines and educational levels [5, 23]. One such high-stakes context that utilizes competitive programming, critical thinking, logical thinking, algorithmic thinking, coding, and more is required. Ranked programming contests, such as ICPC, consist of regional and national qualifying tournaments, leading to the World Finals competition. All programs in ICPC entail codified constructions; i.e., they require well-structured, efficient, and logically sound programming code that adheres to strict input–output and time constraints. The use of GenAI tools on ICPC-style problems presents a promising opportunity to enhance programming education by helping students improve their algorithmic thinking, logical reasoning, and coding skills in a structured and competitive environment.

Every computer science student faces the critical challenge of learning and enhancing their programming skills during their graduation phase. This enables them to utilize GenAI assistants like ChatGPT, DeepSeek, and others for guidance when traditional resources are insufficient. Alarmingly, when students cannot find effective learning resources to develop strong programming foundations, many shift their career interests toward nonprogramming fields such as data analytics, business management, and other domains. This trend motivated us to rigorously investigate the educational efficacy of GenAI tools, especially in competitive programming contexts. But here's the critical question: Which AI tool will actually guide them toward a correct, efficient solution versus leading them down a rabbit hole of syntactically correct but logically flawed code? A significant knowledge gap remains, as no comprehensive comparative study has assessed the effectiveness of GenAI tools on standard programming problems, such as ACM-ICPC problems. This research addresses that gap by systematically comparing ChatGPT and DeepSeek across real ACM ICPC problems, providing students, educators, and institutions with evidence-based insights into which AI tools truly enhance learning outcomes in competitive programming.

2.1 | Coding and AI

For computer science students to thrive in the digital age, both academically and in practical applications that call for technical precision, logical thinking, and algorithmic design, programming abilities are crucial. The landscape of education is evolving due to the emergence of AI, specifically LLMs such as DeepSeek and ChatGPT, among others. By interpreting natural language prompts and generating syntactically and semantically sound code, these AI tools give students dynamic feedback and support. ChatGPT provides flexible explanations and problem-solving techniques suitable for students of varying skill levels, as it has been trained on a diverse range of text and code datasets [7]. In contrast, DeepSeek utilizes fine-tuning on quantities specific to programming to deliver improved performance for more specialized technical challenges [24]. But the first generations of AI-written code were especially from models such as ChatGPT, which remain to be investigated in detail. A primary worry is the validity of the produced code. Models based on AI, trained on massive datasets from specific sources, can generate code that is flawed or does not follow best practices. This is further compounded by the possibility that these models produce syntactically valid code, but it is incorrect in its meaning, so that the running application will crash or behave unpredictably. Maintainability is also a key challenge when it comes to AI-generated code. AI-generated code is frequently not as clear or as well-documented as human developers tend to be, making it hard to understand and to modify [3]. Platforms like GeeksforGeeks have recognized the growth of AI coding assistants in aiding students in practicing and comprehending programming logic through contest-level problem generation and real-time tutoring interfaces, as well as in debugging. Motivated by these advancements, our study aims to evaluate the usefulness of two top AI tools, ChatGPT and DeepSeek, in advanced programming education by evaluating their real performance on extremely challenging ACM ICPC problems.

2.2 | Overview of ACM—ICPC

The ACM, founded in 1947, is the world's foremost scientific and educational society in computing, with a mission to advance the computing profession through publications, conferences, special interest groups, and educational resources. Among its flagship global initiatives is the ICPC, a globally renowned and highly prestigious algorithmic programming competition for university students. The ICPC serves as a platform for fostering innovation, problem-solving skills, and collaboration in real-time, high-pressure environments. It is structured into three main tiers: local contests, where universities select their top teams; regional contests, which feature participants from broader geographic zones; and the culminating world finals, which host only the top 1% of teams worldwide, representing over 3000 universities across more than 100 countries.

In this contest, teams of three students work collaboratively on a single computer to solve 8–13 algorithmic problems within a 5-h time frame. These problems vary in difficulty and test a wide range of technical skills including data structures (trees, heaps, disjoint sets), algorithms (graph theory, dynamic programming, greedy approaches, backtracking), mathematics (combinatorics, number theory, geometry), computational geometry, bit manipulation, and advanced topics such as network flows, line sweep algorithms, and segment trees. Programming languages include C, C++, and Java, with C++ being the most prevalent choice due to its fast execution and rich standard library (STL). Each year, over 50,000 students participate across the ICPC, but only a select few reach the elite world finals. Winners of the world finals are awarded not only global recognition and trophies but also internship offers, job opportunities, and monetary prizes from top tech companies, including IBM, JetBrains, and Huawei. Some editions have awarded cash prizes up to $15,000 for the top team, along with medals, certificates, and often direct fast-track recruitment interviews with prestigious organizations. As thousands of students prepare annually for the ACM ICPC, the lack of dedicated, structured resources for contest-specific training remains a significant gap. Given this context, the potential of GenAI models to support such preparation, by offering real-time feedback, enhancing algorithmic thinking, and improving code precision, demands rigorous empirical evaluation. One recent study presents a systematic survey of computer science education research on algorithm design, highlighting the scarcity of rigorous studies and providing insights into teaching strategies, student engagement, and underutilized algorithmic techniques in undergraduate courses [25]. This underscores the

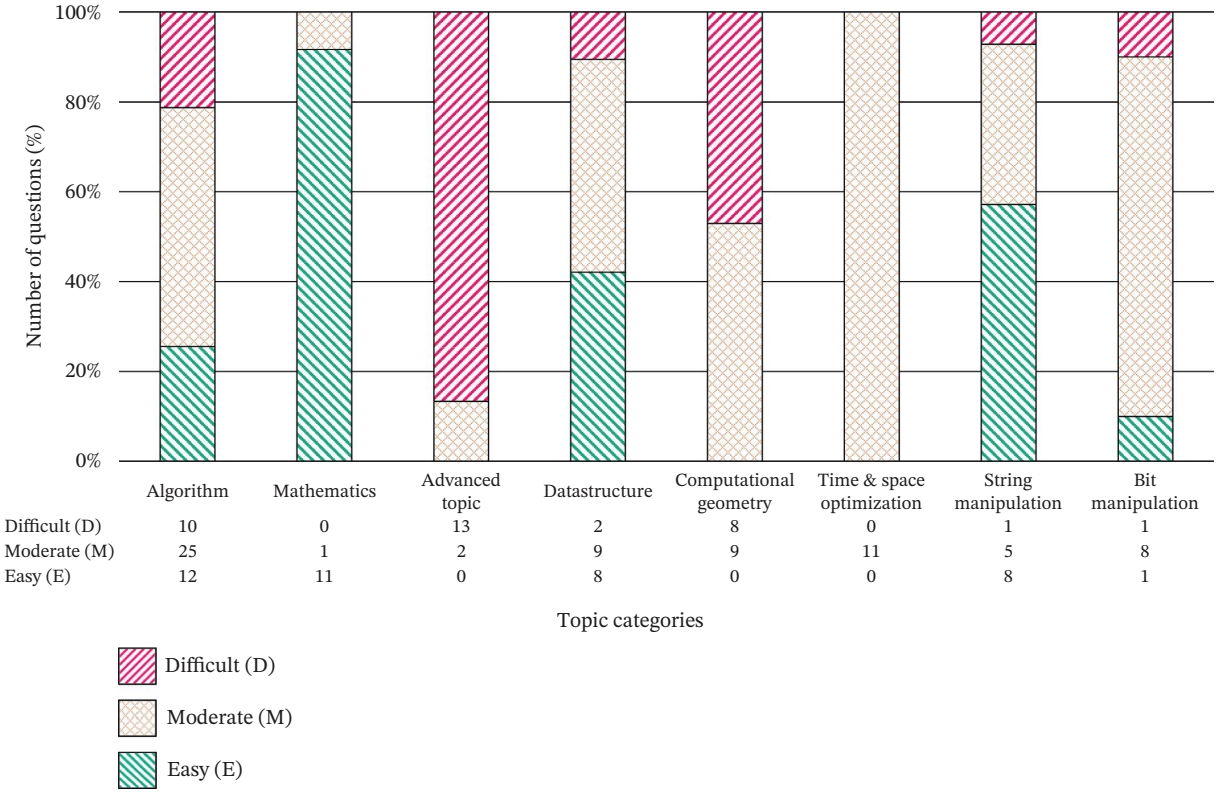

FIGURE 1 | Categorization of questions based on topics and difficulty level.

relevance and timeliness of our comparative study, which aims to assess the practical utility of ChatGPT and DeepSeek in meeting the unique challenges of competitive programming education.

3 | Methodology

This study evaluates two independent variables: (1) the performance of ChatGPT and (2) the performance of DeepSeek on ICPC-style programming problems. For each model, we collected solutions to 145 problems and analyzed them using a set of research tools. Accuracy, readability, and density of insight (DOI) were assessed using a rubric-based evaluation. Time and space complexities were computed using the Big O Calculation Tool, while syntax, logical, and runtime errors were manually verified through compilation and test-case execution. Since the study uses AI-generated outputs and publicly available ICPC problems, no human participants were involved. The methodology has been structured to clearly show how each measurement relates to independent variables.

3.1 | Dataset and Prompting

We gathered and prepared a dataset of 145 programming problems from the official ACM ICPC archives, spanning the period from 2013 to 2024, which covers more than 10 years. We then thoroughly evaluate these problems using ChatGPT and DeepSeek. Based on the historical success rate and time-to-solve information supplied in contest metadata, each problem was initially divided into three standard tiers: easy, moderate, and difficult. The dataset consists of 40, 70, and 35 easy, moderate, and difficult questions, respectively. The questions were then further categorized into topical domains, as illustrated in Figure 1. These included Data Structures, Algorithms, Mathematics, String and Bit Manipulation, Computational Geometry, Advanced Topics, and Optimization Criteria. The model's comprehension of hierarchical relationships, memory management, and the application of union-find optimizations is tested through data structures, specifically problems involving trees, heaps, and disjoint sets. Dynamic programming, graph traversal (Depth First Search/Breadth First Search), greedy strategies, and backtracking tasks in the algorithms category exposed subtle model behaviors. Here, the iterative prompting approach may introduce feedback bias, as repeated corrections could lead to excessively improved model outputs. AI assistance delivery shows that whether support is provided proactively or only upon request can significantly influence user dependence and decision-making [26]. This suggests that prompt delivery style in our study may not only affect model performance but also mirror cognitive engagement patterns observed in human–AI interactions.

In the initial trial, we created a standardized prompt template for submitting problems to both models to ensure evaluation consistency. The official problem statement, expected input format, and constraints were clearly outlined in the prompt. Each model received the same input free from formatting bias through this methodical approach. A second round of testing was carried out for problems that had incorrect answers in the first trial, because of logical mistakes, inefficiencies, or failure to compile. Based on the nature of the prior failure, a revised prompt template was created in this phase to incorporate indirect training. We were able to replicate a learning and feedback loop that students frequently encounter during practice due to this iterative interaction. For both evaluation trials, the outputs generated by

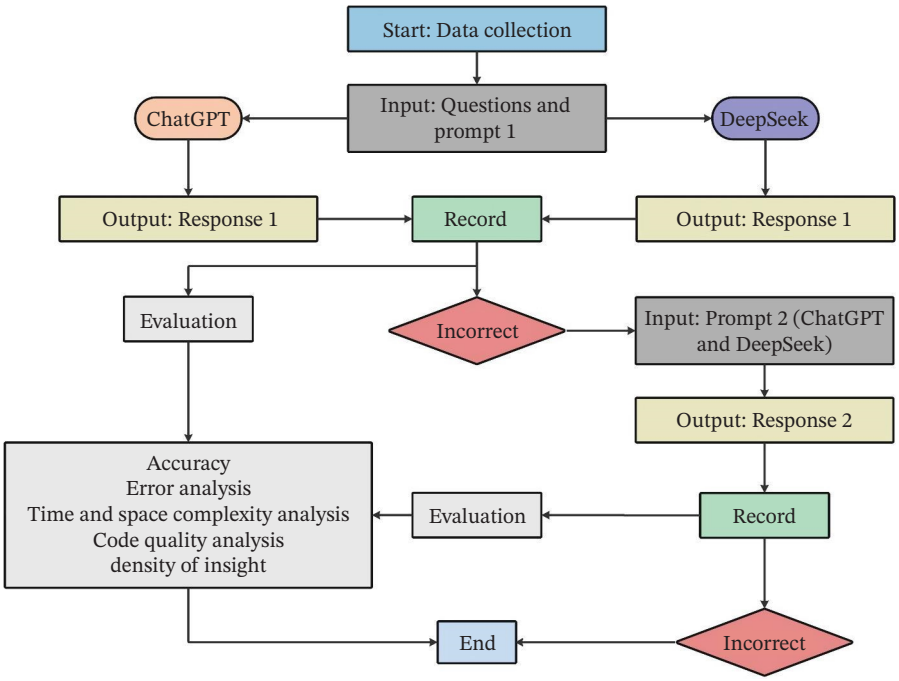

FIGURE 2 | Flowchart demonstrates the dual trial framework adopted for evaluating code generated by ChatGPT and DeepSeek on ACM ICPC problems.

ChatGPT and DeepSeek were systematically documented in a structured tabular format [27]. Each entry captured relevant metadata, including problem ID, difficulty level, topical category, trial number, and key performance indicators such as execution success, logical correctness, and code quality metrics. Code correctness was rigorously verified not only through sample input/output cases provided in the original problem statements but also cross-checked against the official solution sketches and reference implementations published by ACM ICPC.

3.2 | Performance and Quality Metrics

This study employs a comprehensive multidimensional evaluation framework, as shown in Figure 2, to assess the performance and quality of AI-generated code solutions through both quantitative and qualitative metrics. The primary quantitative measure is accuracy, which serves as a fundamental benchmark for problem-solving capability and comparative analysis against other AI systems. Computational complexity analysis is conducted using the Big O notation to evaluate the time and space complexity, providing insights into algorithmic efficiency of the code. Error analysis encompasses three critical categories: runtime errors (execution failures due to memory overflow or invalid operations that compromise system stability), syntax errors (compilation-stage violations of language rules that prevent program execution), and logical errors (algorithmic reasoning defects resulting in incorrect outputs despite successful compilation and execution). Code quality assessment integrates four readability submetrics to generate a composite score: function size, indentation score (percentage of correctly formatted code blocks), identifier quality (percentage of meaningful variable names), and comment density (percentage of commented code lines). Additionally, theoretical explanation quality is evaluated on a five-point scale considering output correctness, code correctness, compendiousness, validity, and nonobviousness. These integrated qualitative and quantitative metrics ensure that generated solutions not only demonstrate functional correctness but also adhere to established software engineering principles, thereby facilitating code maintenance, collaborative development, and educational applications in professional programming environments.

3.3 | Experimental Setup

To ensure consistency and fairness in evaluation, we employed a manual process for inputting and testing prompts across both ChatGPT and DeepSeek models. A standardized prompt format was created for the initial trial (before training), which was uniformly applied to all 145 ACM ICPC problems. Prompt 1 that was used before training: *"PROVIDE C++ CODE, SOLUTION SKETCH EXPLAINING THE APPROACH AND SAMPLE OUTPUT FOR THE GIVEN PROBLEM."* This prompt was followed by the official problem statement as provided in the contest archives. If the output provided by ChatGPT or DeepSeek is incorrect or suboptimal, then we performed indirect training using a second prompt, which is *"THE CODE IS GIVING THE WRONG OUTPUT FOR PROBLEM, REWRITE."* This prompt aimed to guide models toward correction without introducing human bias or providing additional hints, which we referred to as an "indirect prompt". Here, we did not utilize fine-tuning or reinforcement learning, which involves updating a model's internal parameters using additional training data or feedback signals. In this study, iterative prompting along with indirect training is implemented, which is a prompt-level refinement process in which outputs are improved through revised instructions without modifying the model parameters. This approach is therefore distinct from fine-tuning or reinforcement learning, which directly alter model weights. All responses were collected, executed, and analyzed manually to preserve the integrity of the evaluation.

The research was conducted on an HP EliteBook laptop equipped with an Intel Core i5 processor, offering a base frequency of 1.4 GHz and Turbo Boost capability up to 5.0 GHz. The system was supported by 32 GB of RAM and featured PCIe NVMe SSD storage ranging from 512 GB to 1 TB, ensuring fast data access and high-performance multitasking. The device included a 14-inch display with either WUXGA (1920 × 1200) or Full HD (1920 × 1080) resolution, providing clear visual output. For graphics processing, it utilizes Intel UHD Graphics and AMD Radeon Graphics.

For the purpose of experimentally recorded research, all responses generated by ChatGPT and DeepSeek through two iterative trials were meticulously noted. Each of the 145 ACM ICPC problems was randomly scrutinized based on the provided performance and quality metrics to ascertain the suitably generated code and solutions. Compiler responses were noted to check for syntactic correctness and runtime behavior, and the code output was executed in a standardized C++ compilation setup to ascertain uniformity. To determine functional correctness, all outputs were compared against the official sample outputs provided with the problem sets. Iterations tracked any advances or reverses with regard to code correctness and quality by means of the same procedure for both the initial and improved prompt trials. Hence, the double-trial regime combined with quantitative metric analysis gave a thorough yet objective assessment of the two models' code generation abilities under real contest settings.

4 | Results

In the comparative analysis, it was found that DeepSeek outperformed ChatGPT across multiple dimensions of code generation on 145 ACM ICPC problems. DeepSeek demonstrated superior accuracy than ChatGPT (88.28% vs. 84.14%), lower error rates, particularly in logical consistency, and more optimal algorithmic performance, reflected by a higher proportion of O (1) and O (n) solutions. Interestingly, both ChatGPT and DeepSeek consistently produced more readable and well-documented code, as evidenced by approximately the same code quality scores. Additionally, DeepSeek achieved greater instructional depth, as confirmed by the maximum number of fully insightful DOI metric. Although ChatGPT exhibited substantial improvements after iterative refinement, DeepSeek maintained a clear performance advantage before and after training. The following subsections provide a detailed comparative evaluation of ChatGPT and DeepSeek.

4.1 | Accuracy Analysis

Accuracy is defined as the proportion of correct answers given by the GenAI model out of the total questions input to the model. The percentage of correctly answered questions with successfully executed code compared to the total number of questions posed was used to calculate the accuracy of the two evaluated LLMs [5]. DeepSeek consistently outperformed ChatGPT in both experimental trials, according to the comparative analysis. ChatGPT's accuracy before training was 77.93%, while DeepSeek's was 84.14%, a 6.21 percentage point difference in favor of DeepSeek as seen in Figure 3(a). Both models performed better after model optimization after the training. ChatGPT's accuracy increased to 84.14%, matching DeepSeek's initial trial baseline performance. In the second trial, DeepSeek made even more progress, achieving an accuracy of 88.28%, or 4.14 percentage points. These results show that DeepSeek maintains a 4 to 6 percentage point accuracy advantage, even though both models gain a great deal from optimization. From this observation, we can say that ChatGPT's second trial accuracy and DeepSeek's first trial result were equal, indicating that DeepSeek has a naturally higher ability to generate accurate code and solve computational problems in the tested scenarios. On the other hand, it is worth noting that ChatGPT demonstrates more efficient learning than DeepSeek, achieving a 6.21 percentage point improvement after training, compared to DeepSeek's 4.14 percentage point improvement. Accuracy in percentage is calculated using the following equation:

$$\text{accuracy}(\varrho) = \frac{\text{number of correct answers}}{\text{total number of questions}} \times 100. \tag{1}$$

4.2 | Error Analysis

A programming error is a state that violates the specification [28]. Three main types of errors were included in the error rate analysis: syntax, logical, and runtime errors. As shown in Figure 3(b), it is evident that both ChatGPT and DeepSeek produce high logical errors in code, followed by syntax and runtime errors. Syntax errors are mistakes in the structure of a program's code that prevent it from being compiled or executed. It is the most common category of errors in any program. ChatGPT generated syntax errors in 4.83% number of solutions before training, which dramatically decreased to 0.69% after training, signifying a 4.14 percentage point improvement. DeepSeek, on the other hand, improved syntax accuracy by only 2.07 percentage points, reducing the number of syntax errors from 4.83% of the solutions before training to 2.76% after training. A logic error causes incorrect program execution, whereas a syntax error prevents execution altogether. Across successive trials, ChatGPT exhibited a marginal reduction in logical error rate, decreasing it from 13.79% number of solutions to 13.10%. By reducing logical errors from 9.66% to 7.59% of the total number of solutions, DeepSeek demonstrated enhanced logical consistency. A runtime error is an error that occurs during the execution of a program, causing it to terminate unexpectedly. It was found that runtime errors were comparatively stable and minimal in the code provided by both ChatGPT and DeepSeek. While DeepSeek exhibited runtime errors in 1.38% of the solutions before and after training, ChatGPT demonstrated a slight improvement, reducing the runtime error from 1.38% to 0.69% in the total number of solutions after training. There is a complex behavior exhibited by both ChatGPT and DeepSeek in generating errors in code and subsequently improving it after training. Overall, from Figure 3(b), it is observed that while ChatGPT reduced syntax and runtime errors more effectively, DeepSeek reduced logical errors more efficiently. Figure 1 (supporting) shows the total number of syntax, logical, and runtime errors found in the solution acquired from ChatGPT and DeepSeek before and after training. The following equations are used to calculate the syntax, logical, and runtime errors in percentage, respectively.

$$\text{Syntax error}(\varrho) = \frac{\text{Number of syntax errors}}{\text{Total number of questions}} \times 100, \tag{2}$$

$$\text{Logical error}(\varrho) = \frac{\text{Number of logical errors}}{\text{Total number of questions}} \times 100, \tag{3}$$

$$\text{Runtime error}(\varrho) = \frac{\text{Number of runtime errors}}{\text{Total number of questions}} \times 100. \tag{4}$$

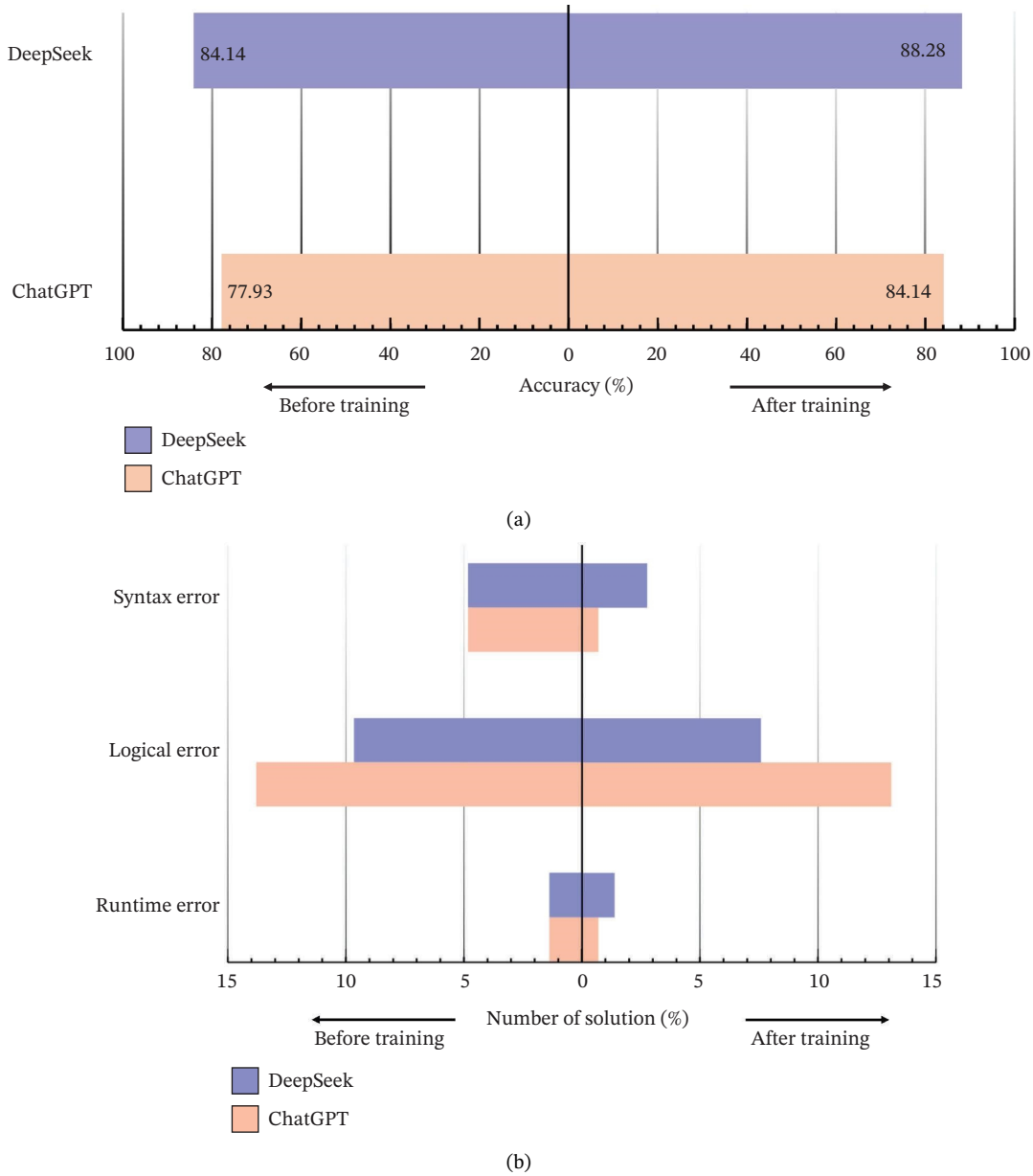

FIGURE 3 | (a) Comparison of the accuracy of ChatGPT and DeepSeek before and after training. (b) Occurrence of syntax, logical, and runtime errors before and after training in ChatGPT and DeepSeek code generation.

4.3 | Time and Space Complexity Analysis

Time complexity refers to the computational time an algorithm requires to execute as a function relative to the input size, while space complexity measures the amount of memory utilized during execution [29]. Both time and space complexity are usually expressed using Big O notation. The Big O calculation tool was used to determine the time and space complexity of solutions obtained from both ChatGPT and DeepSeek for 145 questions. Figure 4 shows that when compared to ChatGPT, DeepSeek generated more constant time O (1) solutions (7 vs. 6) and linear time O(n) solutions (40 vs. 31), quadratic time O(n^2) solutions (47 vs. 19), and cubic O(n^3) solutions (24 vs. 11) before training, suggesting a preference for more efficient algorithms. After training, DeepSeek showed a decrease in higher-complexity outputs like cubic time O(n^3) solutions (4 vs. 9), and a rise in-constant time solutions (8 vs. 6) and linear time solutions (41 vs. 19). Interestingly, DeepSeek's quadratic solutions complexity dropped from 47 before training to 12 after training. Conversely, ChatGPT performed worse after training, showing a rise in complexity classes 12 to 26 and a decrease in linear solutions from 31 to 19. These findings underline DeepSeek's superior capability in producing scalable and optimal code, especially in scenarios requiring stringent performance under competitive programming constraints.

In terms of space complexity, as observed in Figure 5, both ChatGPT and DeepSeek frequently generated solutions with higher space complexity despite improvements after prompt refinement. A significant portion of output from both models fell into complex categories, such as quadratic, exponential, and factorial space, which are generally less desirable in competitive programming contexts. In the initial trial, ChatGPT produced 83 complex-space solutions, while DeepSeek generated 80; after training, these values

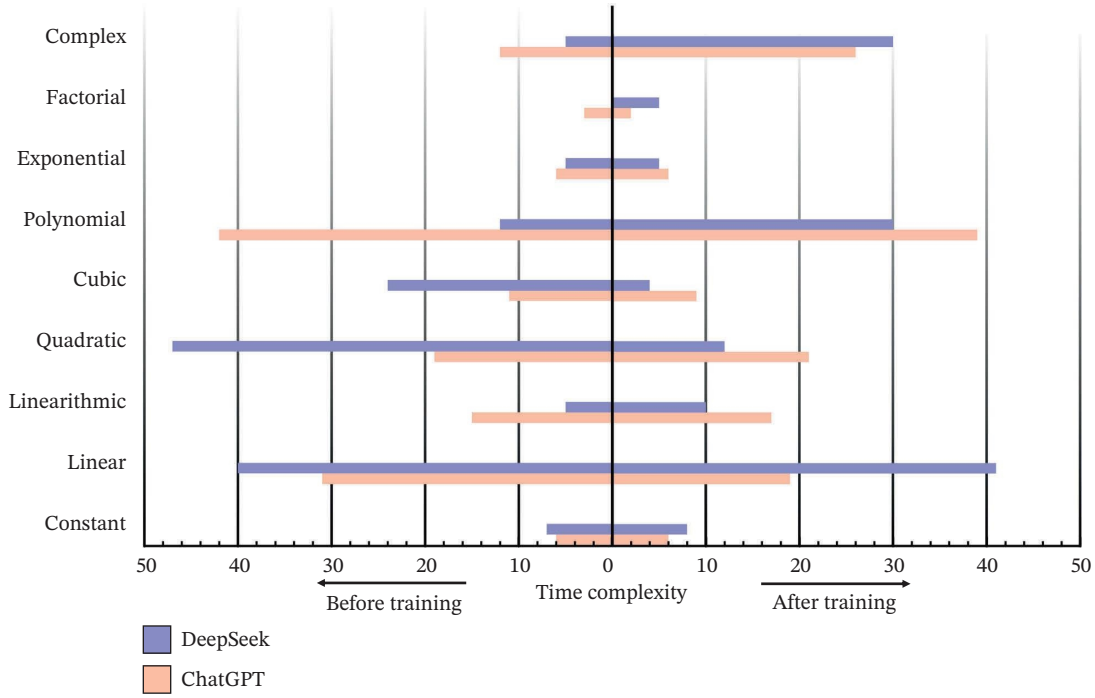

FIGURE 4 | Comparison of nine types of time complexities found in the coding solution obtained from ChatGPT and DeepSeek before and after training. (Complexity classes: constant [O(1), execution time-independent of input size]; linear [O(n), execution time grows proportionally to input size]; linearithmic [O(n log n), execution time grows proportional to n times log n, typical of efficient sorting algorithms]; quadratic [O(n^2), execution time grows quadratically, typically nested loops over input data]; cubic [O(n^3), execution time grows cubically, triple nested operations]; polynomial [O(n^k), execution time depends on degree greater than three]; exponential [O(2^n), execution time doubling with each additional input]; and factorial [O(n!), growth of execution time is based on permutations]. Complex category represents solutions containing multiple asymptotic growth terms within the same algorithm, reflecting mixed computational behaviors [e.g., O(m log $m + m$ log n)]).

remained high at 78 and 81, respectively. This indicates that both models, while capable of solving the problems, often relied on less memory-efficient approaches. However, after training, DeepSeek generated a higher number of quadratic-space O(n^2) and linear-space O(n) solutions, 10 and 32, respectively, while ChatGPT generated 9 and 40, respectively, while significantly reducing occurrences of higher-complexity classes such as cubic, polynomial, exponential, and factorial. These findings highlight the need for further refinement in guiding GenAI models toward space-optimized solution strategies.

4.4 | Code Quality and Readability Analysis

Four subparameters, comment density, identifier quality, indentation score, and function size, were used to calculate the overall code quality [30]. Comment density is defined as the number of lines containing meaningful comments and is calculated by the following equation (5):

$$\text{Comment density}(\varrho) = \frac{\text{Number of comment lines}}{\text{Totol number of lines in code}} \times 100. \quad (5)$$

An identifier is the name assigned to variables, functions, classes, constants, or any other named entity in programming code. Number of identifiers help in determining code's complexity, function size, and maintainability, indicating density of a code. A smaller number of identifiers suggest simple or unclear code, while too many may reduce readability or indicate poor modularization in small code. In this work, a number of identifiers are calculated for each solution obtained from ChatGPT and DeepSeek before and after training. Thereafter, identifier quality is calculated using the following equation:

$$\text{identifier quality} = \frac{\text{number of identifier}}{\text{maximum number of identifier}} \times 100. \quad (6)$$

Indentation is defined as the consistency of indentation throughout the code using spaces or tabs at the beginning of lines of code to visually structure the code into blocks. It helps indicate the logical hierarchy and flow of a program, especially within conditionals, loops, functions, and classes. The indentation score is calculated using the following equation:

$$\text{indentation score}(\varrho) = \frac{\text{number of proerly indented blocks}}{\text{total number of blocks}} \times 100. \quad (7)$$

Function size is the number of functions in a code, favoring modular functions, and it refers to the length and complexity of a function or method in a code. In this work, function size is normalized using the following equation:

$$\text{function size} = \frac{\text{number of functions}}{10} \times 100. \quad (8)$$

The overall readability score is a quantitative measure used to evaluate how easy it is to read and understand code. In this work, we calculated the overall readability score using comment density, identifier quality, indentation score, and function size score by using the following equation:

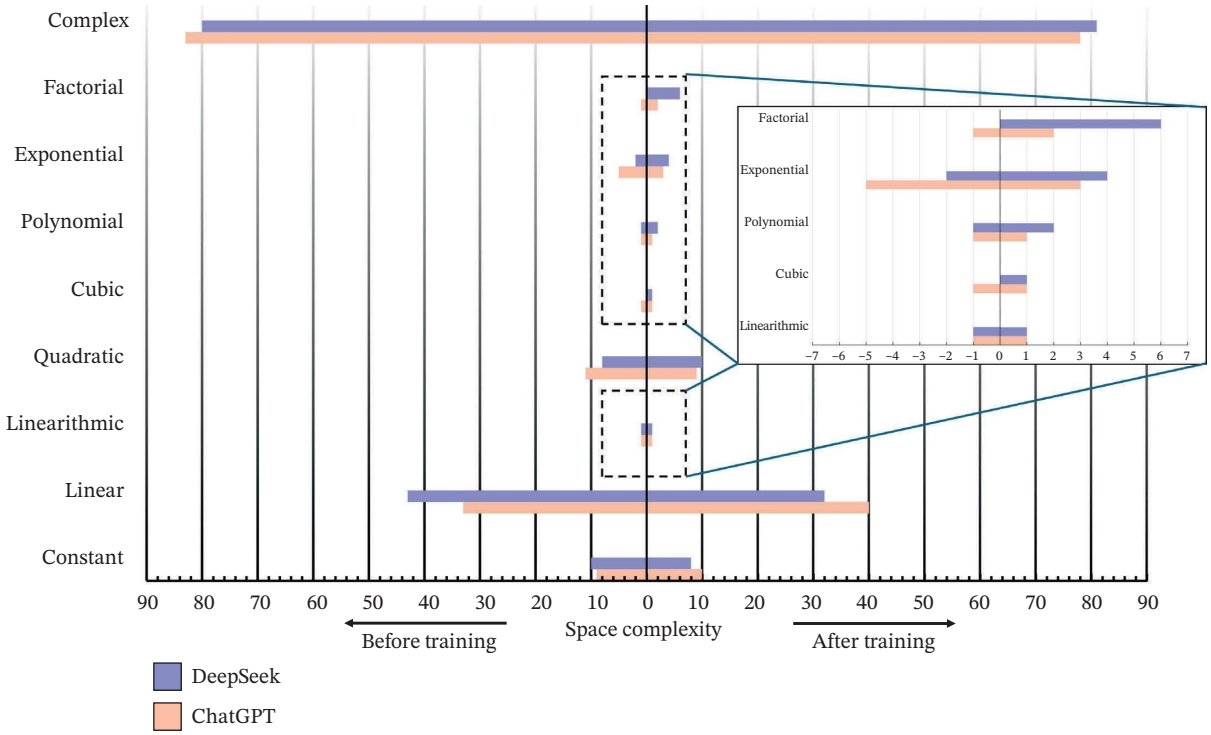

FIGURE 5 | Comparison of nine types of space complexities found in the coding solution obtained from ChatGPT and DeepSeek before and after training. (Complexity classes: constant [O(1), fixed memory usage]; linear [O(n), memory usage proportional to input size]; linearithmic [O(n log n), memory usage grows proportional to n times log n]; quadratic [O(n^2), memory growth with nested data structures]; cubic [O(n^3), representing memory growth with triple nested operations]; polynomial [O(n^k), degree greater than three]; exponential [O(2^n), memory doubling with each additional input]; and factorial [O(n!), memory scales with all permutations]. Complex category represents solutions combining multiple space growth behaviors in the same implementation, capturing mixed memory usage patterns [e.g., O(m log $m + m$ log n)]).

$$\text{overall readability score} = \frac{\text{CD} + \text{IQ} + \text{IS} + \text{FS}}{4}, \tag{9}$$

where CD, IQ, IS, and FS are comment density, identifier quality, indentation score, and function size, respectively.

We calculated average comment density in both GenAI tools, ChatGPT and DeepSeek, before and after training. ChatGPT shows an average comment density of 3.29 and 3.39 before and after training. On the other hand, DeepSeek shows an average comment density of 3.06 and 3.45 before and after training. Maximum comment density is shown by ChatGPT about 40.48 stating ChatGPT has potential in providing very well-documented program code. Next, we calculated identifier quality by considering small/medium code blocks. The maximum number of identifiers is provided by DeepSeek, which is 64, which is considered as base to calculate identifier quality in this work. ChatGPT showed an average identifier quality of 26.01 and 26.78, while DeepSeek provided 26.67 and 29.11 before and after training, respectively. This clearly demonstrates that the codes provided by DeepSeek are readable, maintainable, and understandable as compared to ChatGPT.

Both models exhibited exceptional performance in code formatting, achieving perfect 100% indentation scores across all trials, indicating outstanding adherence to structural coding standards. On the other hand, average function size metrics showed significant differences: ChatGPT's outputs were relatively verbose, with function sizes of 20.62 before training and 21.17 after training, whereas DeepSeek produced more concise code, with function sizes of 17.38 and 18.62 before and after training, respectively. Lastly, we calculated the overall readability score. ChatGPT achieved average overall readability scores of 37.48 and 37.85, while DeepSeek scored 36.78 and 37.79 for the same tasks, respectively.

Collectively, DeepSeek outperformed in all four subparameters average comment density, identifier quality, indentation score, and function size. Also, DeepSeek's outputs were consistently more readable and maintainable, according to additional analysis of identifier naming and structural organization. Even though both models got better after training and have approximately the same overall readability score, ChatGPT can be considered the better tool for producing production-grade code because of its higher baseline performance, achieving an overall readability score of 37.85.

4.5 | DOI Analysis

The DOI analysis is used to assess the generated outputs' instructional quality and explanatory depth. The DOI is defined as the ratio of the number of insights available in the explanation to the total number of insight criteria, i.e., DOI = (Number of insights)/(Total number of insight criteria) [5]. Figure 6 illustrates the DOI scores for both models, ChatGPT and DeepSeek, before and after training, across the range of values from 0 to 1. On a scale of 0–1, DOI values were calculated using five criteria: output correctness, code correctness, compendiousness, validity, and nonobviousness. While DOI scores below 1 indicated insight deficiency, a score of 1 indicated fully insightful

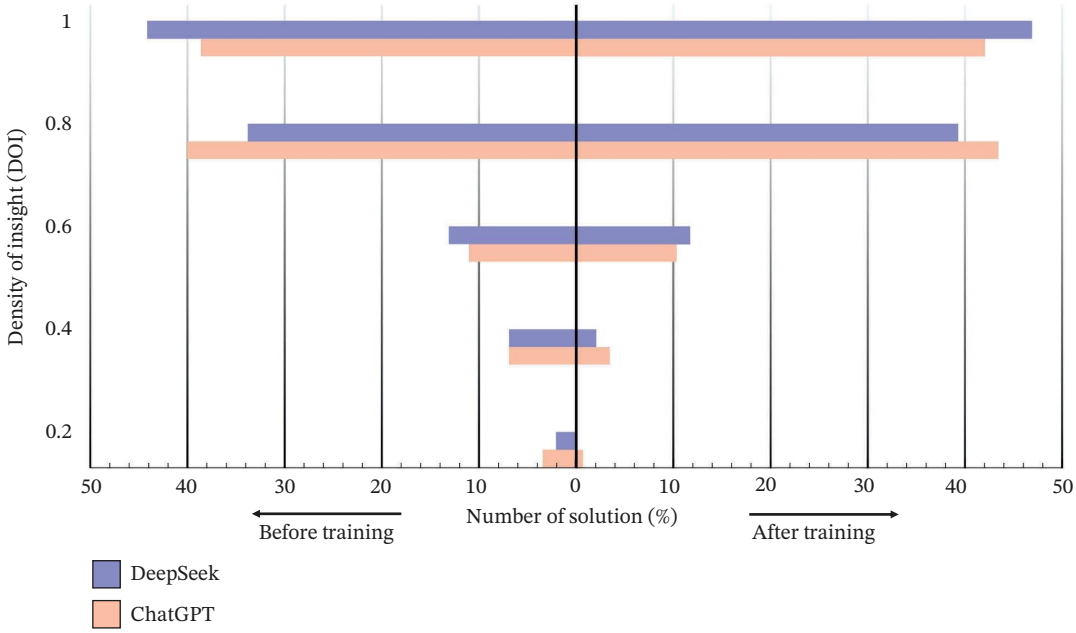

FIGURE 6 | Density of insight calculated in the solution acquired from ChatGPT and DeepSeek before and after training.

content. DeepSeek performed better in terms of DOI before and after training. DeepSeek beat ChatGPT's 38.62% before training and 42.07% after training in the highest insight category (DOI = 1), achieving 44.14% before training and 46.90% after training. Both ChatGPT and DeepSeek produced null solutions with no insight (DOI = 0). Also, it should be emphasized that there is complex behavior when compared to the medium level of insight categories. These findings imply that DeepSeek continuously produced solutions that were more perceptive and educationally useful, covering algorithmic justification and solution methodology more thoroughly. The DOI analysis emphasizes how DeepSeek's outputs have additional instructional utility, which increases its efficacy for use cases involving instruction and explanation.

5 | Discussion

In this work, we identified the strengths and weaknesses of two GenAI tools, ChatGPT and DeepSeek, in complicated learning and problem-solving scenarios on programming tasks at the ACM ICPC level. DeepSeek usually had greater accuracy and better performance than ChatGPT on most metrics, although ChatGPT also showed significant benefits in flexibility and iterative development. The advantage of domain-specific fine-tuning is reflected in the supremacy of DeepSeek, indicated by its higher accuracy and algorithmic solution ratings. DeepSeek outperformed ChatGPT in generating efficient code, producing 8 constant-time O(1) and 41 linear-time O(n) solutions after training, while ChatGPT's linear-time solutions dropped from 31 to 19 and its higher-complexity outputs increased. For space complexity, both models still produced many memory-intensive solutions, but DeepSeek improved with 32 linear-space O(n) and 10 quadratic-space O(n^2) solutions, compared to ChatGPT's 40 linear-space and 9 quadratic-space outputs. Overall, these results indicate that DeepSeek is more effective in producing scalable, optimized code, although both models still require refinement to achieve memory-efficient solutions. Production of code that is more time- and space-optimized by means of a higher count of O(1) and O(n) solutions and fewer complex O(n^2) or O(n^3) algorithms shows that it stands well with the competitive programming perspective. Additionally, by maintaining a lower logical error rate before and after training, DeepSeek appeared to have a firmer grasp of algorithmic structure and computational logic. However, in contrast to the two-trial setting, ChatGPT displayed remarkable flexibility. Of interest here is how much the model improved over time, dropping syntax errors from 4.83% to 0.69% of the number of solutions, a greater decrease than DeepSeek. This suggests that the quick improvement and high sensitivity to fast optimization are ChatGPT's strengths. Along with this, ChatGPT demonstrates more efficient learning than DeepSeek, showing a 6.21 percentage point improvement after training compared to DeepSeek's 4.14 percentage point improvement.

Furthermore, due to its flexibility and a larger training corpus, ChatGPT provided responses for various types of problems, including some that DeepSeek sometimes failed to tackle, such as complex string manipulations. DeepSeek had the advantage with respect to generating code with good readability and structural clarity, with better function modularity, identifier quality, and comment density. Nevertheless, ChatGPT's ability to give explanations in natural language is truly remarkable. ChatGPT's clear and detailed explanations made its outputs easier for beginners to understand, making it especially useful for those just starting to learn programming, even though DeepSeek provided more technically detailed solutions. While DeepSeek scored higher on the DOI metric, indicating more in-depth suggestions, ChatGPT's output still offered significant educational value through simpler, beginner-friendly explanations. This means that there exists a complementary use of these two models in the learning space: one for ChatGPT, with respect to range and pedagogical clarity, and DeepSeek for depth and technical accuracy. In conclusion, ChatGPT's flexibility, quick development, and user-friendly results confirm its applicability in educational

support systems, even though DeepSeek's unique architecture gives it a distinct edge in competitive programming scenarios. The potential for hybrid instructional models that utilize both general-purpose and domain-specific AI tools to enhance programming education is evident from their combined strengths. Insights from adjacent domains, such as cybersecurity and digital twin technologies, show that AI tools must be evaluated not only for accuracy but also for adaptability and reliability within complex systems [31]. This viewpoint strengthens our discussion by suggesting that structured comparisons of GenAI models, like those in this study, can also guide evaluations in other complex technological systems.

6 | Conclusion

This study compared ChatGPT and DeepSeek on 145 ICPC-level problems to determine how each model supports programming reasoning and code generation. This work explains that GenAI tools, specifically for the purpose of programming, have a lot of potential to improve in competitive programming instruction and training. Iterative prompt refinement helps both ChatGPT and DeepSeek in providing improved outputs. DeepSeek continuously outperforms ChatGPT in important performance metrics like solution accuracy, complexity optimization, error resilience, code readability, and instructional insight. We found that ChatGPT excels in generating accessible explanations and broader problem coverage, while DeepSeek consistently produces more structured, readable, and performance-oriented solutions. Together, these results answer our research questions by showing that the two models offer complementary strengths: one supports conceptual learning and early-stage preparation, while the other aligns more closely with competitive algorithmic thinking. These results highlight how useful it is to integrate these models into learning environments for subjects that require a lot of algorithms, especially when specialized assistance is required. The competitive advantage of DeepSeek implies that domain-specific fine-tuning is essential for forming AI capabilities in niche settings such as competitive coding. This study emphasizes the significance of critical model evaluation and customized application as educational institutions and students increasingly rely on AI for support. Particularly in high-stakes situations like the ACM ICPC, DeepSeek, in its current configuration, stands out as a very capable model for assisting programming pedagogy. The accuracy, time, and space complexity, readability, and DOI analysis can help instructors select model-generated examples that match students' skill levels. Learners preparing for ICPC can strategically rely on ChatGPT for conceptual understanding and DeepSeek for practicing efficient, competition-oriented code. Compared with recent literature, our findings extend current understanding by demonstrating how GenAI tools differ not only in output quality but also in the cognitive support they provide for learners. However, more training, either in the form of indirect or direct, will be beneficial in improving the output from GenAI tools.

7 | Limitations and Future Work

Even though this study is thorough, there are a few limitations to consider. One major limitation is that currently, neither DeepSeek nor ChatGPT offers metadata about response generation time. The ability to evaluate model response latency would provide important insights into their real-time applicability in competitive programming contexts where time efficiency is critical. A more comprehensive evaluation framework that considers both code quality and assistance speed may be made possible by future models or API implementations that incorporate timestamped output generation. The evaluation's exclusive use of the C++ programming language is another drawback. Java and Python, which are extensively used in professional and educational contexts, could provide a more generalizable understanding of each model's capabilities across various programming paradigms, even though C++ is the most common language in ACM ICPC contests. Next, the use of automated tools, such as Big O calculators and rubric-based ratings, introduces measurement constraints that may not fully capture algorithmic nuances. Also, the absence of human participants prevents us from linking model outputs to actual learning outcomes, such as student cognitive flexibility, AI anxiety, or self-regulated learning behaviors identified in prior studies. Moreover, the dataset comprises past ICPC problems, some of which may have been present in the training corpora of ChatGPT or DeepSeek. Consequently, the models may have had partial prior exposure, which could contribute to elevated performance on certain problems.

Additionally, the study can thoroughly examine the impact of prompt engineering in the future, even though it used a uniform prompt template to guarantee equity. The impact of different prompt structures, instructions, and contextual framing on model performance could be methodically examined in future research. For educators and developers looking to optimize the use of AI coding assistants, knowing how various prompt designs affect output accuracy, efficiency, and explanation depth may offer crucial insights. Furthermore, broadening the focus to incorporate real-time collaborative environments, like AI-assisted hackathons or pair programming simulations, would better reflect real-world use cases. Comparative robustness would be improved, and new AI tools in this field could be benchmarked by incorporating other emerging LLMs designed for code generation, such as Code Llama or StarCoder. To sum up, this study offers a strong framework for assessing GenAI models in competitive programming; however, it can be expanded in the future by incorporating response latency tracking, multilanguage support, sophisticated prompt engineering, and real-time educational integration. The incorporation of these models into dynamic, iterative real-time development environments is another possible direction for future research. Additional evidence of these models' educational value may be provided by monitoring long-term learning outcomes when students use them for guided problem-solving and by gathering qualitative input from educators and users.

Author Contributions

Harshita Vyas: data curation, analysis, visualization, and writing–original draft. Ravindra G. Bhardwaj: conceptualization, analysis, supervision, methodology, visualization, validation, and writing–review and editing.

Funding

The authors received no specific funding for this work.

Disclosure

All authors read and approved the final manuscript. An earlier version of this manuscript was made available as a preprint [22]. The current version has been substantially revised and expanded based on further analysis and peer review feedback.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The datasets used and/or analyzed during the current study are available from the corresponding author upon reasonable request.

References

1. S. Ehtesham, H. S. Bedi, and R. G. Bhardwaj, "Performance Evaluation of ChatGPT on BITSAT: Engineering Entrance Examination," *Hybrid Intelligent Systems* (Dubai: Cham, 2025).

2. J. Varghese and J. Chapiro, "ChatGPT: The Transformative Influence of Generative AI on Science and Healthcare," *Journal of Hepatology* 80, no. 6 (2024): 977–980, https://doi.org/10.1016/j.jhep.2023.07.028.

3. S. Almanasra and K. Suwais, "Analysis of ChatGPT-Generated Codes Across Multiple Programming Languages," *Institute of Electrical and Electronics Engineers* 13 (2025): 23580–23596, https://doi.org/10.1109/ACCESS.2025.3538050.

4. W. F. Godoy, P. Valero-Lara, K. Teranishi, P. Balaprakash, and J. S. Vetter, "Large Language Model Evaluation for High-Performance Computing," *Software Development* 36, no. 26 (2024): e8269, https://doi.org/10.1002/cpe.8269.

5. R. G. Bhardwaj and H. S. Bedi, "ChatGPT as an Education and Learning Tool for Engineering, Technology and General Studies: Performance Analysis of ChatGPT 3.0 on CSE, GATE and JEE Examinations of India," *Interactive Learning Environments* 33, no. 1 (2025): 321–334, https://doi.org/10.1080/10494820.2024.2344054.

6. R. Yilmaz and F. G. Karaoglan Yilmaz, "Augmented Intelligence in Programming Learning: Examining Student Views on the Use of ChatGPT for Programming Learning," *Computers in Human Behavior: Artificial Humans* 1, no. 2 (2023): 100005, https://doi.org/10.1016/j.chbah.2023.100005.

7. A. Bucaioni, H. Ekedahl, V. Helander, and P. T. Nguyen, "Programming With ChatGPT: How Far Can We Go?" *Machine Learning With Applications* 15 (2024): 100526, https://doi.org/10.1016/j.mlwa.2024.100526.

8. Z. Deng, W. Ma, Q. L. Han, et al., "Exploring DeepSeek: A Survey on Advances, Applications, Challenges and Future Directions," *IEEE/CAA Journal of Automatica Sinica* 12, no. 5 (2025): 872–893, https://doi.org/10.1109/JAS.2025.125498.

9. Y. Xu, X. Liu, X. Cao, et al., "Artificial Intelligence: A Powerful Paradigm for Scientific Research," *Innovation* 2, no. 4 (2021): 100179, https://doi.org/10.1016/j.xinn.2021.100179.

10. R. Shakya, F. Vadiee, and M. Khalil, "A Showdown of ChatGPT vs DeepSeek in Solving Programming Tasks," in *International Conference on New Trends in Computing Sciences (ICTCS)* (Amman, Jordan, April 2025).

11. S. Nikolic, D. Scott, H. Rezwanul, et al., "ChatGPT Versus Engineering Education Assessment: A Multidisciplinary and Multi-Institutional Benchmarking and Analysis of This Generative Artificial Intelligence Tool to Investigate Assessment Integrity," *European Journal of Engineering Education* 48, no. 4 (2023): 559–614, https://doi.org/10.1080/03043797.2023.2213169.

12. M. Jordan, K. Ly, and A. G. S. Raj, "Need a Programming Exercise Generated in Your Native Language? ChatGPT's Got Your Back: Automatic Generation of Non-English Programming Exercises Using OpenAI GPT-3.5," in *ACM Technical Symposium on Computer Science Education* (March, 2024), https://doi.org/10.1145/3626252.3630897.

13. A. Y. Q. Huang, C.-Y. Lin, S.-Y. Su, and S. J. H. Yang, "The Impact of GenAI-Enabled Coding Hints on Students' Programming Performance and Cognitive Load in an SRL-Based Python Course," *British Journal of Educational Technology* 56, no. 5 (2025): 1942–1972, https://doi.org/10.1111/bjet.13589.

14. J. Liu, S. Poulsen, E. Goodwin, et al., "Teaching Algorithm Design: A Literature Review," *ACM Transactions on Computing Education* 25, no. 2 (2025): 1–20, https://doi.org/10.1145/3727987.

15. M. Y. H. Low, C. C. Lee, K. K. Lee, and K. L. Lam, "Enhancing the Teaching of Data Structures and Algorithms Using AI Chatbots," in *IEEE International Conference on Teaching, Assessment and Learning for Engineering (TALE)* (Bengaluru, India, December, 2024).

16. M. Zimmermann, H. Janetzko, and B. Haymond, "Integrating Generative AI Methods in Computer Science Education: Perspectives, Strategies, and Outcomes," in *16th International Conference on Education and New Learning Technologies* (Palma, Spain, July, 2024), 10358–10365, https://doi.org/10.21125/edulearn.2024.2546.

17. C. Cubillos, R. Mellado, D. Cabrera-Paniagua, and E. Urra, "Generative Artificial Intelligence in Computer Programming: Does It Enhance Learning, Motivation, and the Learning Environment?" *Institute of Electrical and Electronics Engineers* 13 (2025): 40438–40455, https://doi.org/10.1109/ACCESS.2025.3532883.

18. U. A. Shuvo, S. A. Dip, N. R. Vaskar, and A. B. M. A. A. Islam, "Assessing Chatgpt's Code Generation Capabilities With Short vs Long Context Programming Problems," in *11th International Conference on Networking, Systems, and Security* (Khulna, Bangladesh, January, 2025), 32–40, https://doi.org/10.1145/3704522.3704535.

19. S. Vishnu, Sahil, and N. Garg, "Unveiling the Role of GPT-4 in Solving LeetCode Programming Problems," *Computer Applications in Engineering Education* 33, no. 1 (2025): e22815, https://doi.org/10.1002/cae.22815.

20. Y. Zheng and M. Sarem, "C++ Teaching Reform and Exploration Based on ACM/ICPC and Live Code," in *2022 5th International Conference on Education Technology Management* (Lincoln, UK, May, 2022).

21. J. J. Blum, "Competitive Programming Participation Rates: An Examination of Trends in U.S. ICPC Regional Contests," *Discover Education* 2, no. 1 (2023): 11, https://doi.org/10.1007/s44217-023-00034-1.

22. H. Vyas and R. G. Bhardwaj, "ChatGPT vs DeepSeek: A Comparative Evaluation on the International Computer Science Benchmark–ACM ICPC" (July 2025), https://doi.org/10.21203/rs.3.rs-7077588/v1.

23. S. Hajkowicz, C. Sanderson, S. Karimi, A. Bratanova, and C. Naughtin, "Artificial Intelligence Adoption in the Physical Sciences, Natural Sciences, Life Sciences, Social Sciences and the Arts and Humanities: A Bibliometric Analysis of Research Publications From 1960–2021," *Technology in Society* 74 (2023): 102260, https://doi.org/10.1016/j.techsoc.2023.102260.

24. I. Jin, J. A. Tangsrivimol, E. Darzi, et al., "DeepSeek vs. ChatGPT: Prospects and Challenges," *Frontiers in Artificial Intelligence* 8 (2025): 1576992–2025, https://doi.org/10.3389/frai.2025.1576992.

25. X. Liu, X. Sun, L. Bo, Y. Hu, X. Liu, and Z. Ye, "Evaluating the Test Adequacy of Benchmarks for LLMs on Code Generation," *Journal of Software: Evolution and Process* 37, no. 7 (2025): e70034, https://doi.org/10.1002/smr.70034.

26. P. P. M. Nair, G. Gressel, M. N. Anand, and K. Achuthan, "To Solicit or Not to Solicit? Impact of AI Assistance Delivery Mechanisms on Decision-Making," *International Journal of Human-Computer Interaction* (2025): 1–24, https://doi.org/10.1080/10447318.2025.2536617.

27. H. B. Vyas and Ravindra, "Data Set-ChatGPT vs DeepSeek: A Comparative Evaluation on the International Computer Science Benchmark–ACM ICPC" (2025), https://doi.org/10.17632/4ccc4dhpms.1.

28. J. Laski, "Programming Faults and Errors: Towards a Theory of Software Incorrectness," *Annals of Software Engineering* 4, no. 1 (1997): 79–114, https://doi.org/10.1023/A:1018966827888.

29. Q. Gao and X. Xu, "The Analysis and Research on Computational Complexity," in *26th Chinese Control and Decision Conference (2014 CCDC)* (Changsha, China, June 2014).

30. Y. Feng, S. Vanam, M. Cherukupally, W. Zheng, M. Qiu, and H. Chen, "Investigating Code Generation Performance of ChatGPT With Crowdsourcing Social Data," in *IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)* (Torino, Italy, June 2023).

31. K. Achuthan, B. B. Gupta, and R. Raman, "Bridging Cybersecurity With Digital Twin Technology: A Thematic Analysis," *International Journal of Information Security* 24, no. 5 (2025): 207, https://doi.org/10.1007/s10207-025-01115-y.

Supporting Information

Additional supporting information can be found online in the Supporting Information section. *(Supporting Information)*

Supporting file "ACM ICPC Supporting File.docx" is provided.

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.